PuppyRaffle Smart Contract Security Audit Report

Prepared by: Cyfrin
Lead Auditor: KAJO
Date: 17 October 2026
Version: 1.0

Table of Contents

- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
    - Scope
    - Roles
- Executive Summary
    - Issues found
- Findings
    - High
    - Medium
    - Low
    - Informational
    - Gas

Protocol Summary

The **PuppyRaffle** protocol is designed as an on-chain lottery mechanism operating on Ethereum. Users buy tickets by depositing an entrance fee to enter a randomized drawing pool. The final winner is drawn to receive an allocation of the total ticket prize pool alongside unique Puppy NFTs, while a percentage of the total entry cost is strictly accumulated as protocol revenue fees.

Disclaimer

The Cyfrin team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

Scope

- **Commit Hash:** `2a47715b30cf11ca82db148704e67652ad679cd8`
- **In Scope Files:**

Roles

- **Lead Auditor:** KAJO

Executive Summary

Issues found

Severity	Count	Status
High	3	Unresolved
Medium	1	Unresolved
Low	1	Unresolved
Gas	2	Unresolved
Info/NC	1	Unresolved

Findings

High

[H-01] Reentrancy Vulnerability in `refund` Allows Entrants to Drain Contract Balance

Description:
The `PuppyRaffle::refund` function fails to adhere to the **Checks-Effects-Interactions (CEI)** pattern. The contract performs an external transfer of funds to the player via `sendValue` *before* modifying the contract state to update the `players` array.

```
function refund(uint256 playerIndex) public {
    address playerAddress = players[playerIndex];
    require(playerAddress == msg.sender, "PuppyRaffle: Only the player can refund");
    require(playerAddress != address(0), "PuppyRaffle: Player already refunded, or is not a

    // Interaction: External transfer happens before the state change
    payable(msg.sender).sendValue(entranceFee);
```

```
    // Effect: State update happens too late
    players[playerIndex] = address(0);

    emit RaffleRefunded(playerAddress);
}
```

An attacker can register an entrance using a malicious contract containing a `receive()` or `fallback()` function. When the attacker calls `refund`, the contract sends the `entranceFee`. This triggers the attacker's fallback function, allowing them to re-enter the `refund` function repeatedly before the player's index is reset to `address(0)`.

Impact:
A malicious actor can exploit this vulnerability to recursively withdraw funds, completely draining the entire native currency balance of the `PuppyRaffle` contract.

Proof of Concept (PoC):

1. Multiple legitimate users enter the raffle, funding the contract balance.
2. The attacker deploys a malicious contract (`ReentrancyAttacker`) designed with a fallback mechanism that intercepts incoming transfers.
3. The attacker enters the raffle using the attacking contract and calls `attack()` to initiate the first refund, looping execution and draining the remaining contract balance.

▶ View Executable Foundry Test Code

Recommended Mitigation:
Update the state variable (`players[playerIndex] = address(0)`) *before* executing the external transfer:

```
function refund(uint256 playerIndex) public {
    address playerAddress = players[playerIndex];
    require(playerAddress == msg.sender, "PuppyRaffle: Only the player can refund");
    require(playerAddress != address(0), "PuppyRaffle: Player already refunded, or is not a

    players[playerIndex] = address(0); // Effect
    payable(msg.sender).sendValue(entranceFee); // Interaction
    emit RaffleRefunded(playerAddress);
}
```

[H-02] Permanent Lock of Protocol Revenue in `withdrawFees` via Forced Ether Injection

Description:
The `withdrawFees` function relies on a strict equality check between the contract's actual balance and the accumulated fee tracking state variable:

```
require(address(this).balance == uint256(totalFees), "PuppyRaffle: There are currently pl
```

In Ethereum, relying on `address(this).balance` for strict routing check paths is unsafe. An attacker can force Ether into the `PuppyRaffle` contract through mechanisms that bypass standard function execution, such as calling `selfdestruct(target)` from a separate contract.

Impact:
Once the contract's balance is forced to be even 1 Wei higher than `totalFees`, the condition `address(this).balance == uint256(totalFees)` will permanently evaluate to false. This results in a permanent Denial of Service (DoS) on revenue extraction, locking all fees forever.

Proof of Concept (PoC):

1. `PuppyRaffle` accumulates `10 ETH` in `totalFees`.
2. An attacker deploys a dummy contract funded with `1 Wei` and calls `selfdestruct(puppyRaffleAddress)`.
3. The `PuppyRaffle` contract balance is forced to `10 ETH + 1 Wei`.
4. The admin invokes `withdrawFees()`, which permanently reverts.

Recommended Mitigation:
Validate if there are players left via internal state trackers (e.g., checking if the players array length is empty) instead of checking the native balance configuration:

```
function withdrawFees() external {
    require(players.length == 0, "PuppyRaffle: There are currently players active!");
    uint256 feesToWithdraw = totalFees;
    totalFees = 0;
    (bool success,) = feeAddress.call{value: feesToWithdraw}("");
    require(success, "PuppyRaffle: Failed to withdraw fees");
}
```

[H-03] Weak Randomness in `selectWinner` Allows Attackers to Predict or Influence Results

Description:
The `PuppyRaffle::selectWinner` function utilizes predictable environmental variables such as `msg.sender`, `block.timestamp`, and `block.difficulty` to compute the random winner sequence via hashing. On-chain values are inherently deterministic, and validators can see or manipulate these values ahead of time.

Impact:
Malicious users or validators can pre-calculate the outcome of the raffle, choosing the exact block timing or context to guarantee selection. This lets them claim the rarest rewards and steal the main prize pool.

Recommended Mitigation:
Avoid generating randomness via local block variables. Integrate an asynchronous verifiable off-chain oracle mechanism such as **Chainlink VRF (Verifiable Random Function)**.

Medium

[M-01] Denial of Service (DoS) via Unbounded Loop in `enterRaffle`

Description:
The `PuppyRaffle::enterRaffle` function utilizes a nested `for` loop to scan the entire `players` storage array to ensure that no duplicate addresses are entered into the raffle during execution.

```
for (uint256 i = 0; i < players.length - 1; i++) {
    for (uint256 j = i + 1; j < players.length; j++) {
        require(players[i] != players[j], "PuppyRaffle: Duplicate player");
    }
}
```

Impact:
As the `players` array grows larger, the gas cost of checking for duplicates scales quadratically ($O(N^2)$). Eventually, the gas cost will exceed the block gas limit, causing all new entries to revert and effectively bricking the contract.

Recommended Mitigation:
Instead of scanning the array with a nested loop, use a `mapping(address => bool)` to track registered players in $O(1)$ constant time complexity.

Low

[L-01] Missing Access Control Modifier on `withdrawFees`

Description:
The `withdrawFees` function is configured as `external` but lacks any restriction constraints like `onlyOwner`. Although funds are transferred directly to `feeAddress`, allowing un-authenticated accounts to trigger administrative accounting sequences is an architectural vulnerability.

Impact:
Unnecessary external exposure increases the contract's attack surface and allows malicious actors to front-run or disrupt expected protocol operations.

Recommended Mitigation:
Introduce an ownership validation check parameter (e.g., `onlyOwner` modifier or `require(msg.sender == feeAddress)`).

Informational

[I-01] Use of Magic Numbers Decreases Code Readability

Description:
The codebase directly utilizes raw numerical literals ("magic numbers") for percentage formulas and pool distributions instead of declaring clear constant identifiers.

```
uint256 prizePool = (totalAmountCollected * 80) / 100;
uint256 fee = (totalAmountCollected * 20) / 100;
```

Recommended Mitigation:
Define expressive `constant` state variables at the contract level to handle precision arithmetic:

```
uint256 public constant PRIZE_POOL_PERCENTAGE = 80;
uint256 public constant FEE_PERCENTAGE = 20;
```